Trabajo Práctico 6 — Resolución

Aprendizaje Automático

Inteligencia Artificial — UNS — 2026

Las fuentes van en notas al pie. **Cátedra**: diapositivas de A. J. García (2026). **AIMA**: Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 3.ª ed. (2010). **PMG**: Poole, Mackworth & Goebel, *Computational Intelligence: A Logical Approach* (1998).

Índice

Ejercicio 1 — Aprendizaje **2**

1.a . . . 2

1.b . . . 3

1.c . . . 3

1.d . . . 4

Ejercicio 2 — Medición de la performance **6**

Ejercicio 3 — Árboles de Decisión **8**

3.a . . . 8

3.b . . . 9

3.c . . . 10

3.d . . . 11

Ejercicio 4 — Redes Neuronales Artificiales (RNA) **13**

4.a . . . 13

4.b . . . 14

4.c . . . 16

4.d . . . 17

Ejercicio 1 — Aprendizaje

1.a

Consigna

¿Cómo se define el Aprendizaje Automático (Machine Learning)? ¿En qué contextos se aplica? ¿Qué disciplinas científicas confluyen en él?

- **Definición**: área de la IA que estudia y construye algoritmos que, a partir de datos de entrenamiento:[1]
  1. reconocen automáticamente patrones en los datos,
  2. construyen un modelo, y
  3. con ese modelo reconocen o hacen predicciones sobre datos no vistos en el entrenamiento.
- **Contextos**: problemas en los que diseñar y programar a mano un algoritmo explícito con buen rendimiento es difícil o imposible.[2] Ejemplos de la cátedra:
  - reconocer señales de velocidad máxima en imágenes;
  - reconocer dígitos escritos a mano;
  - un asistente que sugiere qué impresora usar.
- **Disciplinas**: está dentro de la IA y usa herramientas de la **estadística** y de las **ciencias de la computación**.[3]

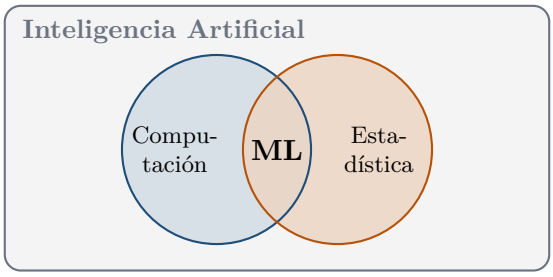


Figura 1: El aprendizaje automático está dentro de la IA, entre la estadística y la computación.

EXTRAS

- El término *machine learning* lo acuñó Arthur Samuel en 1959.[4]
- En AIMA:[5]
  - una definición centrada en el agente: "*An agent is learning if it improves its performance on future tasks after making observations about the world.*"
  - otras dos razones para que un agente aprenda en lugar de recibir todo programado: los diseñadores no pueden prever todas las situaciones (un robot tiene que aprender el plano de cada laberinto nuevo) ni los cambios con el tiempo (un predictor de la bolsa tiene que adaptarse cuando el mercado pasa de auge a crisis).
- Algunas técnicas de ML: árboles de decisión, redes neuronales, *deep learning*, reglas de asociación, redes bayesianas, *clustering*, algoritmos genéticos, SVM, programación lógica

[1] Cátedra, Episodio VII, parte 1, «Machine Learning – aprendizaje automático».
[2] Cátedra, Episodio VII, parte 1, «Aprendizaje automático (machine learning)». Misma idea en AIMA, cap. 18, p. 693 (reconocer caras: "*even the best programmers are unable to program a computer to accomplish that task except by using learning algorithms*").
[3] Cátedra, Episodio VII, parte 1, «Aprendizaje automático (machine learning)».
[4] Cátedra, Episodio VII, parte 1, «Aprendizaje automático (machine learning)».
[5] AIMA, cap. 18, p. 693.

inductiva.[6]

1.b

Consigna

¿Cuáles son las tres etapas del proceso de aprendizaje para la creación de un modelo?

Son tres etapas:[7]

1. **Elección (trabajo humano)**: los especialistas eligen el algoritmo de entrenamiento y el conjunto de datos de entrenamiento.
2. **Creación automática del modelo**: el algoritmo recorre los ejemplos del conjunto de entrenamiento, busca patrones y ajusta el modelo automáticamente.
3. **Evaluación (trabajo humano)**: los especialistas deciden si el resultado es satisfactorio, es decir, si el modelo sirve para la tarea (reconocer personas, generar texto, etc.).

Etapa	Quién	Entrada	Resultado
1	especialistas	tarea a resolver	algoritmo y datos de entrenamiento elegidos
2	algoritmo (automático)	datos de entrenamiento	modelo ajustado
3	especialistas	modelo	decisión: ¿es un buen modelo para la tarea?

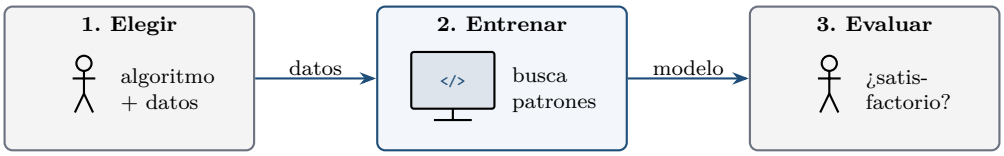


Figura 2: Etapas de la creación de un modelo: humano, automático, humano.

EXTRAS

- Lo que se entrena es el modelo. Los algoritmos los diseñan de antemano equipos de científicos; el software que los implementa hace el trabajo computacional de entrenar.[8]
- Los datos de entrenamiento son *el combustible* del aprendizaje: el resultado muchas veces es tan bueno como los datos. Pueden ser imágenes, audio, video, texto o datos tabulares (una columna por atributo).[9]

1.c

Consigna

¿A qué nos referimos con el término "modelo" en el contexto de Machine Learning?

- **Modelo, en general**: una representación simplificada de un objeto, sistema, fenómeno o proceso de la realidad, para comprenderlo, analizarlo, explicarlo, predecirlo o simularlo.[10]

[6] Cátedra, Episodio VII, parte 1, «(Algunas) áreas de Inteligencia Artificial».
[7] Cátedra, Episodio VII, parte 1, «Modelo».
[8] Cátedra, Episodio VII, parte 1, «Modelo».
[9] Cátedra, Episodio VII, parte 1, «Los datos de entrenamiento».
[10] Cátedra, Episodio VII, parte 1, «Modelo».

- **Modelo, en ML**:[11]
  - es el conjunto de reglas o patrones aprendidos;
  - es una **estructura matemática** que representa la relación entre las entradas (datos) y las salidas (predicciones);
  - es una abstracción de los datos de entrenamiento que permite **generalizar** y predecir sobre datos nuevos.
- **Quién lo crea**: en otras disciplinas, los modelos los escriben personas (una fórmula, un texto). En ML los **genera un algoritmo** a partir de los datos de entrenamiento.[12]
- **Ejemplos**: un árbol de decisión, o una red neuronal con los pesos de sus arcos (un grafo).[13]

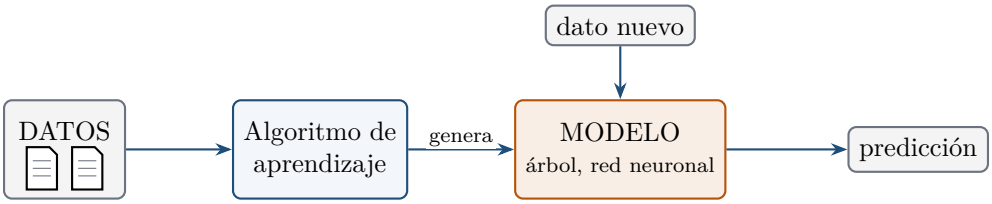


Figura 3: El algoritmo genera el modelo a partir de los datos. Después, el modelo predice sobre datos nuevos.

EXTRAS

- Ejemplo de modelo no aprendido: T = distancia/velocidad. Ignora semáforos, tráfico y estado del camino, pero alcanza para no llegar tarde. Si sabemos que es un modelo, lo usamos con cuidado aunque tenga errores.[14]
- En AIMA el modelo se llama **hipótesis** h: una función que aproxima la función verdadera y desconocida f que generó los ejemplos. "*Learning is a search through the space of possible hypotheses for one that will perform well, even on new examples beyond the training set.*"[15]

1.d

Consigna

Investigue en qué se diferencian los tres paradigmas clásicos de aprendizaje: Supervisado, No Supervisado y por Refuerzo.

La cátedra solo los nombra.[16] Se diferencian por el tipo de **retroalimentación** (*feedback*) que recibe el que aprende:[17]

- **Supervisado**: observa pares entrada–salida (ejemplos **etiquetados** por un "maestro") y aprende una función que lleva de la entrada a la salida. Ejemplo: aprender a reconocer dígitos a partir de imágenes etiquetadas con el dígito.
- **No supervisado**: no recibe ninguna retroalimentación explícita; descubre patrones en la entrada. La tarea más común es el **clustering** (agrupar ejemplos parecidos). Ejemplo: un taxi que, sin que nadie le dé ejemplos etiquetados, forma los conceptos "días de buen tránsito" y "días de mal tránsito".

[11] Cátedra, Episodio VII, parte 2, «Uso del árbol de decisión para clasificación de nuevos elementos».
[12] Cátedra, Episodio VII, parte 1, «Modelo».
[13] Cátedra, Episodio VII, parte 3, «Algoritmo de aprendizaje y modelo».
[14] Cátedra, Episodio VII, parte 1, «Modelo».
[15] AIMA, §18.2, p. 695. Misma idea («learning as search») en PMG, §11.1, p. 401.
[16] Cátedra, Episodio VII, parte 1, «(Algunas) áreas de Inteligencia Artificial».
[17] AIMA, §18.1, pp. 694–695. Misma clasificación en PMG, §11.1, p. 399.

- **Por refuerzo**: aprende a partir de una serie de **recompensas o castigos**, que a menudo llegan después de una secuencia de acciones. El agente tiene que deducir cuáles de sus acciones fueron responsables (*credit-assignment problem*). Ejemplo: el taxi que no recibe propina, o los dos puntos por ganar una partida de ajedrez.

	Supervisado	**No supervisado**	**Por refuerzo**
Retroalimentación	la salida correcta de cada ejemplo	ninguna	recompensa o castigo
Datos	pares entrada–salida (etiquetados)	solo entradas	secuencias de acciones y recompensas
Aprende	función entrada → salida	patrones, grupos (*clustering*)	qué acciones conviene hacer
Ejemplo	dígitos etiquetados	días de buen/mal tránsito	propina, partida ganada

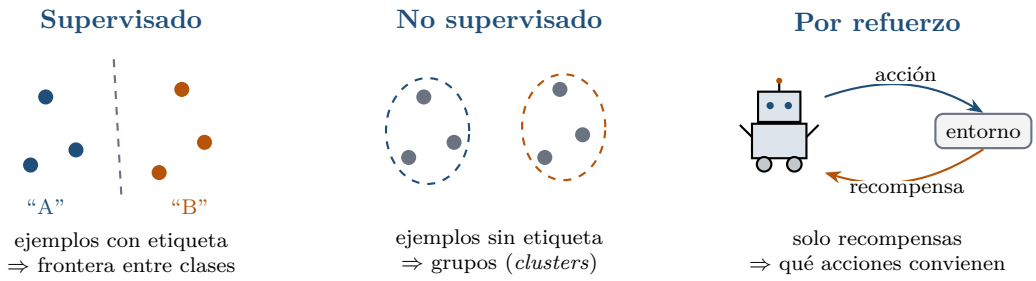


Figura 4: Los tres paradigmas según la retroalimentación que reciben.

EXTRAS

- **Semisupervisado**: pocos ejemplos etiquetados y muchos sin etiquetar. Las etiquetas pueden tener errores sistemáticos (personas que mienten su edad en fotos etiquetadas). Por eso el ruido y la falta de etiquetas forman un continuo entre supervisado y no supervisado.[18]
- PMG distingue el supervisado **de conceptos** (un entrenador da la clase de cada ejemplo) del supervisado **de acciones** (el agente recibe retroalimentación inmediata sobre el valor de cada acción). El refuerzo queda entre el supervisado y el no supervisado.[19]
- Dentro del supervisado: si la salida es un valor de un conjunto finito, es **clasificación** (p. ej. inkjet/laser); si es un número, es **regresión** (p. ej. el precio de una casa).[20]

[18] AIMA, §18.1, p. 695.
[19] PMG, §11.1, p. 399.
[20] AIMA, §18.2, p. 696; Cátedra, Episodio VII, parte 2, «Algoritmos de Aprendizaje con Árboles de Decisión».

Ejercicio 2 — Medición de la performance

Consigna

Explique brevemente para qué se utiliza una métrica en el entrenamiento y en la evaluación. ¿Por qué es importante diferenciar entre el desempeño obtenido para el conjunto de entrenamiento y el obtenido para el conjunto de pruebas? ¿A qué nos referimos al hablar de overfitting del modelo de aprendizaje? ¿Cómo podemos identificarlo?

Métrica. Mide numéricamente qué tan bien predice el modelo (p. ej. porcentaje de aciertos o tasa de error).

- **En el entrenamiento**:[21]
  - guía el ajuste: el algoritmo busca los parámetros que minimizan el error (en una red neuronal, el error cuadrático; ver Ejercicio 4.b);
  - sirve de criterio de corte: se entrena hasta llegar a un error aceptable (p. ej. 97 % de aciertos).
- **En la evaluación**: se calcula sobre datos reservados y sirve para decidir si el modelo es satisfactorio (etapa 3 del Ejercicio 1.b), es decir, cuántos fallos estamos dispuestos a aceptar.[22]

Entrenamiento vs. prueba. Un error bajo sobre el conjunto de entrenamiento no implica que el modelo generalice bien, porque el modelo ya vio esos ejemplos. Lo que importa es cómo predice ejemplos nuevos, y eso se estima con un conjunto de prueba **distinto** del de entrenamiento.[23]

Overfitting (sobreajuste). El modelo se ajusta a distinciones que aparecen en los datos de entrenamiento pero no en los ejemplos no vistos (correlaciones casuales o ruido). Clasifica muy bien el entrenamiento y mal los datos nuevos.[24]

Cómo identificarlo. Se comparan los dos errores mientras crece la complejidad del modelo: hay sobreajuste cuando el error de entrenamiento sigue bajando (o queda en casi cero) mientras que el error sobre datos no vistos empieza a subir.[25]

	Conjunto de entrenamiento	**Conjunto de prueba**
¿El modelo lo vio?	sí	no (está reservado)
Uso de la métrica	guiar el ajuste y decidir cuándo parar	decidir si el modelo es satisfactorio
Qué mide	cuánto se ajustó a esos ejemplos	cuánto generaliza
Con sobreajuste	error muy bajo	error alto

EXTRAS

- Analogía de AIMA: "*A professor knows that an exam will not accurately evaluate students if they have already seen the exam questions.*"[26]

[21] Cátedra, Episodio VII, parte 3, «Neuronas Artificiales».
[22] Cátedra, Episodio VII, parte 3, «Redes Neuronales Artificiales» (¿Cómo se evalúa una RNA?); Episodio VII, parte 1, «Machine Learning – aprendizaje automático».
[23] AIMA, §18.4, p. 708. Misma idea en PMG, §11.1, p. 401.
[24] PMG, §11.2, p. 408. Misma idea en AIMA, §18.3.5, p. 705.
[25] AIMA, §18.4, p. 711.
[26] AIMA, §18.4, p. 708.

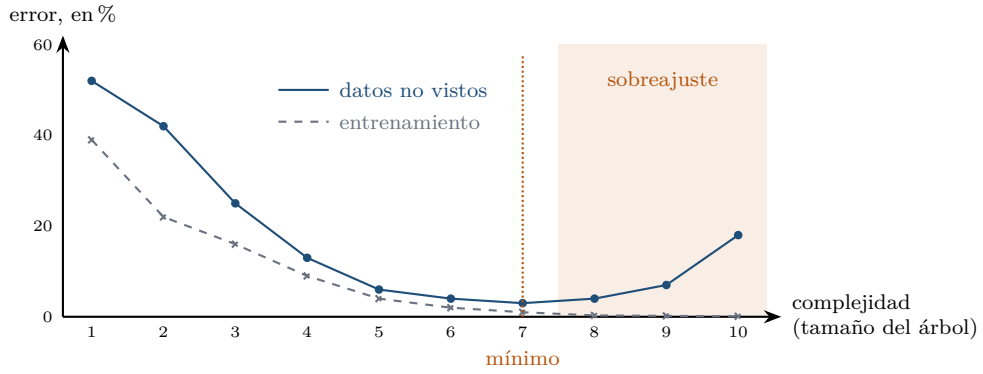


Figura 5: Datos de AIMA, Fig. 18.9: al crecer el árbol, el error de entrenamiento tiende a 0, pero desde el tamaño 7 el error sobre datos no vistos sube.

- Ejemplo de PMG: dos agentes, P y N, clasifican bien todo el entrenamiento. P dice que todo lo no visto es positivo y N, que es negativo: no coinciden en ningún ejemplo nuevo. El desempeño en el entrenamiento no permite elegir entre los dos; hace falta un **sesgo** (*bias*), es decir, una preferencia por ciertas hipótesis que no sale de los datos.[27]
- **Formas de separar los datos**:[28]
  - *holdout*: una sola partición en entrenamiento y prueba. Desperdicia datos, y con pocos datos de prueba la estimación es poco confiable;
  - *k-fold cross-validation*: k rondas, en cada una se reserva $1/k$ de los datos para prueba, y se promedia (valores típicos de k: 5 y 10). Con $k = n$ se llama *leave-one-out*;
  - **conjunto de validación**: usar el error de prueba para elegir el modelo y también para evaluarlo (*peeking*) filtra información de la prueba al aprendizaje. Para elegir se usa un conjunto de validación, y la prueba se guarda "bajo llave" hasta el final.
- No todos los errores cuestan lo mismo: con una **función de pérdida** se puede decir que clasificar como spam un mail legítimo es 10 veces peor que lo contrario. Pérdidas típicas: $L_1 = |y - \hat{y}|$, $L_2 = (y - \hat{y})^2$, $L_{0/1}$ (1 si se equivoca).[29]
- La cátedra muestra los dos tipos de error de un clasificador de señales: **falso positivo** (dice "es señal de velocidad máxima" y no lo es) y **falso negativo** (dice que no y sí lo es).[30]

---

[27] PMG, §11.1, p. 401.
[28] AIMA, §18.4, pp. 708–709.
[29] AIMA, §18.4.2, pp. 710–711.
[30] Cátedra, Episodio VII, parte 1, «Machine Learning – aprendizaje automático».

Ejercicio 3 — Árboles de Decisión

3.a

Consigna

¿Qué es y cuál es la estructura formal de un árbol de decisión? ¿Qué tipo de aprendizaje permite desarrollar? ¿Qué propiedades debe satisfacer un conjunto de datos para poder usarse con un árbol de decisión?

- **Qué es**: un modelo que vincula los atributos relevantes de cada observación con la decisión que se tomó en el conjunto de entrenamiento. Una vez construido, se usa para responder o decidir ante casos nuevos.[31]
- **Estructura formal**: un árbol en el que:[32]
  - los **nodos internos** se etiquetan con atributos de los datos;
  - los **arcos** que salen de un nodo con atributo A se etiquetan con cada uno de los valores posibles de A;
  - las **hojas** se etiquetan con clasificaciones (p. ej. inkjet/laser).
- **Tipo de aprendizaje**: **supervisado**, de **clasificación** (cuando el resultado es una clase). También hay árboles de **regresión**, cuando el resultado es un número real.[33]
- **Propiedades del conjunto de datos**:[34]
  - datos tabulares: todos los ejemplos se describen con los mismos atributos y cada uno trae su clasificación (etiqueta);
  - cada atributo tiene un rango de valores **finito y discreto**;
  - el conjunto tiene que ser **adecuado** (ver 3.b).

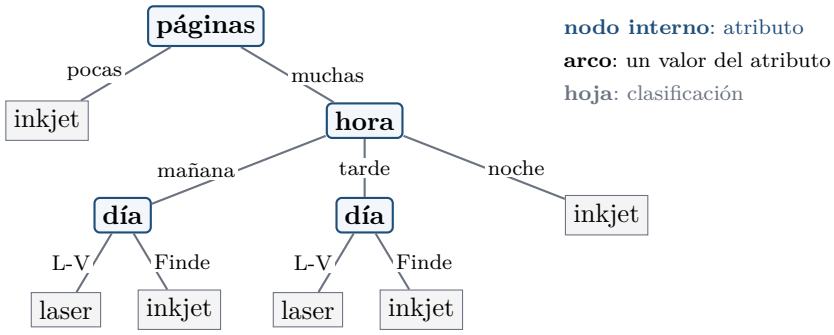


Figura 6: Árbol de decisión de la cátedra para el asistente de impresión.

EXTRAS

- AIMA lo define como una función: recibe un vector de valores de atributos y devuelve una única salida, la "decisión". Admite entradas y salidas continuas; los atributos discretos son el caso más simple.[35]
- Un árbol equivale a un conjunto de reglas, una por hoja: la clase de la hoja vale si se cumplen todas las condiciones del camino desde la raíz. En lógica, $Goal \Leftrightarrow (Camino_1 \vee Camino_2 \vee \cdots)$,

[31] Cátedra, Episodio VII, parte 2, «Árbol de decisión».
[32] Cátedra, Episodio VII, parte 2, «Árbol de decisión». Misma definición en PMG, §11.2, p. 403.
[33] Cátedra, Episodio VII, parte 2, «Aprendizaje por clasificación supervisada: árboles de decisión» y «Algoritmos de Aprendizaje con Árboles de Decisión».
[34] Cátedra, Episodio VII, parte 2, «Árbol de decisión».
[35] AIMA, §18.3.1, p. 698.

en forma normal disyuntiva. Ejemplo con el árbol de la figura: *páginas* = muchas ∧ *hora* = mañana ∧ *día* = L-V ⇒ laser.[36]

- Algoritmos clásicos:[37]
  - **ID3** (Quinlan, 1986): clasificación con variables objetivo categóricas;
  - **CART** (Breiman, Friedman, Olshen y Stone, 1984): clasificación y regresión, con variables objetivo categóricas o numéricas.
- El algoritmo elige primero el atributo más "importante". Una heurística común es la **ganancia de información** (entropía): se parte por el atributo que más reduce la incertidumbre sobre la clase.[38]

3.b

Consigna

¿Qué característica debe tener un conjunto de entrenamiento para ser "adecuado"?

No debe haber **dos ejemplos que coincidan en todos los atributos y tengan distinta clasificación**.[39]

- Ejemplo: si al conjunto de la impresora se le agregara un ejemplo *e15* con los mismos atributos que *e1* pero con decisión *laser*, el conjunto dejaría de ser adecuado. Ningún árbol podría clasificar bien a los dos.

Ejemplo	día	hora	páginas	tipo	tamaño	decisión
e1	L-V	mañana	pocas	pdf	grande	**inkjet**
e15 (hipotético)	L-V	mañana	pocas	pdf	grande	**laser**

Mismos atributos, distinta clasificación ⇒ conjunto no adecuado.

EXTRAS

- El conjunto de la impresora (e1–e14) es adecuado: el árbol de la cátedra clasifica bien los 14 ejemplos.[40]
- Por qué puede pasar: "*there is an error or noise in the data; because the domain is nondeterministic; or because we can't observe an attribute that would distinguish the examples.*" En ese caso AIMA devuelve la clase mayoritaria de esos ejemplos.[41]
- PMG propone otra salida: que las hojas devuelvan una probabilidad en lugar de una clase.[42]

---

[36] PMG, §11.2, pp. 403–404; AIMA, §18.3.2, p. 698.
[37] Cátedra, Episodio VII, parte 2, «Algoritmos de Aprendizaje con Árboles de Decisión».
[38] PMG, §11.2, pp. 407–408; AIMA, §18.3.4, pp. 703–704.
[39] Cátedra, Episodio VII, parte 2, «Árbol de decisión». Misma definición en PMG, §11.2, p. 407.
[40] Cátedra, Episodio VII, parte 2, «Conjunto de entrenamiento (datos tabulares)».
[41] AIMA, §18.3.3, p. 701.
[42] PMG, §11.2, p. 407.

3.c

Consigna

¿Cómo se utiliza un árbol de decisión ya construido para clasificar una nueva observación y qué significado tiene que algunos atributos del conjunto de entrenamiento no figuren en la estructura del árbol?

Clasificar. Se parte de la raíz. En cada nodo se sigue el arco que corresponde al valor que tiene la observación en ese atributo, hasta llegar a una hoja: la etiqueta de la hoja es la clasificación.[43]

- *n1* = (L-V, mañana, muchas, doc, chico): páginas = muchas → hora = mañana → día = L-V → **laser**.
- *n2* = (Finde, noche, muchas, pdf, chico): páginas = muchas → hora = noche → **inkjet**.

Atributos que no figuran. El algoritmo no los necesitó: pudo clasificar todos los ejemplos de entrenamiento sin consultarlos. Según esos datos, son **irrelevantes** para la decisión. En el ejemplo, *tipo* y *tamaño* no influyen en la elección de la impresora.[44]

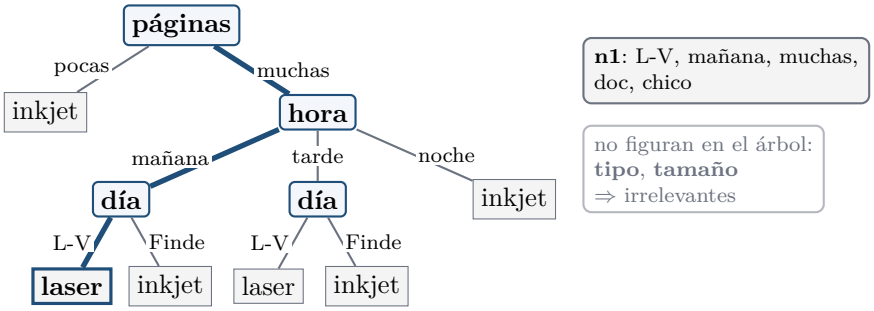


Figura 7: Camino que recorre *n1* hasta la hoja *laser*.

EXTRAS

- Un atributo que está en el árbol puede no consultarse en una observación dada: *n2* llega a una hoja sin pasar por *día*.
- "Irrelevante" vale **según esos datos**, no necesariamente en la realidad. Si hay varios atributos de importancia similar, la elección entre ellos es algo arbitraria: con ejemplos un poco distintos, el árbol puede cambiar mucho de estructura aunque calcule una función parecida. AIMA advierte sobre el peligro de sobreinterpretar el árbol.[45]
- En casos sin ejemplos parecidos, el árbol puede equivocarse. AIMA muestra un caso: el árbol aprendido decide "no esperar" en una situación que nunca vio, y con más ejemplos el error se corregiría.[46]

[43] PMG, §11.2, p. 403; Cátedra, Episodio VII, parte 2, «Uso del árbol de decisión para clasificación de nuevos elementos».
[44] AIMA, §18.3.3, p. 701; Cátedra, Episodio VII, parte 2, «Árbol de decisión» (el árbol vincula los atributos *relevantes* con la decisión).
[45] AIMA, §18.3.3, p. 702.
[46] AIMA, §18.3.3, pp. 701–702.

3.d

Consigna

Investigue por qué los árboles de decisión tienden al sobreajuste (overfitting) cuando crecen sin restricciones.

La cátedra no lo trata.

- **Pueden representar cualquier función** de los atributos, así que nada impide que el árbol memorice el conjunto de entrenamiento.[47]
- **Se parte hasta clasificar todo**: sin restricciones, el algoritmo sigue agregando nodos hasta que el árbol es consistente con todos los ejemplos. Así aprovecha **cualquier patrón** de los datos, incluidas las correlaciones casuales y el ruido, que no se repiten en los datos nuevos.[48]
- **Las hojas profundas se apoyan en pocos ejemplos**: cada partición reduce los ejemplos de cada rama, y una hoja que surge de 2 ejemplos no es evidencia de un patrón real.[49]
  - Ejemplo de AIMA: predecir si un dado sale 6, con atributos irrelevantes (color, peso, hora, dedos cruzados). Lo correcto es un único nodo "no". Pero si dos tiradas de un dado azul de 7 g con los dedos cruzados salieron 6, el algoritmo arma un camino que predice 6 para ese caso.
- Por eso el sobreajuste es más probable cuanto más grande es el espacio de hipótesis y cuantos más atributos hay, y menos probable cuantos más ejemplos de entrenamiento hay.[50]

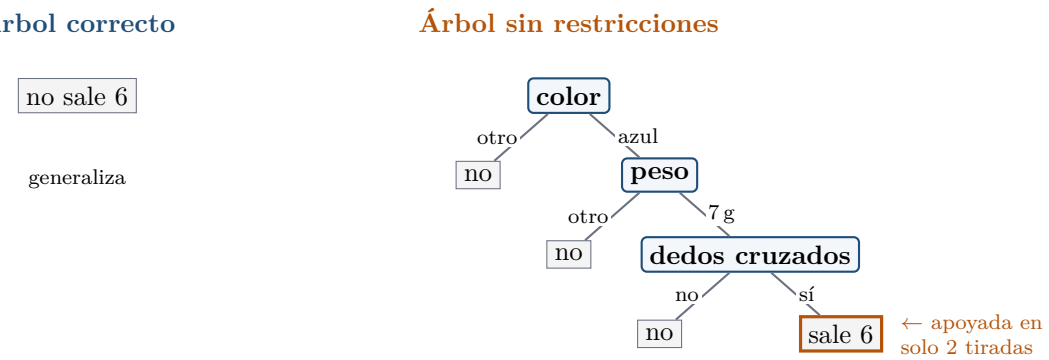


Figura 8: Ejemplo del dado: el árbol sin restricciones memoriza una coincidencia.

	Árbol chico	**Árbol sin restricciones**
Cuándo deja de partir	antes de clasificar todo	cuando clasifica todo el entrenamiento
Ejemplos por hoja	muchos	pocos (a veces 1 o 2)
Qué captura	patrones generales	también coincidencias y ruido
Error en entrenamiento	algo mayor	casi 0
Error en datos nuevos	menor	mayor (ver la figura del Ejercicio 2)

EXTRAS

- **Remedios**: dos formas de combatirlo:[51]
  - **restringir la partición**: partir solo cuando la partición es útil (*early stopping*);

[47] PMG, §11.2, p. 404; AIMA, §18.3.2, pp. 698–699.
[48] AIMA, §18.3.5, p. 705; PMG, §11.2, p. 408.
[49] AIMA, §18.3.5, p. 705.
[50] AIMA, §18.3.5, p. 705.
[51] PMG, §11.2, p. 408; AIMA, §18.3.5, pp. 705–706.

- **podar** (*pruning*): construir el árbol completo y eliminar los nodos que hacen distinciones no justificadas. AIMA usa una prueba de significancia estadística (χ^2): si la partición no se aparta de lo que daría el azar, el nodo se reemplaza por una hoja.

- La poda funciona mejor en la práctica. El *early stopping* falla cuando ningún atributo es útil por sí solo pero sí una combinación (p. ej. XOR de dos atributos): se detendría en la raíz.[52]
- Los árboles podados toleran ruido en los ejemplos y son más chicos, y por lo tanto más fáciles de entender.[53]
- **Random Forest** construye varios árboles de decisión para mejorar la tasa de clasificación.[54]

---

[52] PMG, §11.2, p. 408; AIMA, §18.3.5, p. 706.
[53] AIMA, §18.3.5, p. 706.
[54] Cátedra, Episodio VII, parte 2, «Algoritmos de Aprendizaje con Árboles de Decisión».

Ejercicio 4 — Redes Neuronales Artificiales (RNA)

4.a

Consigna

Describa la estructura de una RNA. Mencione las capas que la componen y las características de cada una.

- **Estructura**: una red de **neuronas artificiales** (NA). Estructuralmente es un **grafo dirigido** de nodos interconectados por arcos.[55]
  - Cada **arco** guarda un **peso** w.
  - Cada **NA** es una función matemática: recibe valores por sus arcos entrantes, calcula un valor con los pesos y lo envía por todos sus arcos salientes.
- **Tipos de NA**:[56]
  - **perceptrón**: devuelve 0 si $w \cdot x + b \leq 0$ y 1 si $w \cdot x + b > 0$ (el sesgo es $b = -\text{umbral}$);
  - **sigmoide**: devuelve $\sigma(w \cdot x + b)$, con $\sigma(z) = \dfrac{1}{1 + e^{-z}}$ (una versión suave del escalón).
- **Capas**: en general está organizada en capas:[57]
  - **de entrada** (*input layer*): recibe el estímulo del exterior de la red. En el reconocimiento de dígitos de 28×28 píxeles tiene 784 neuronas, una por píxel;
  - **ocultas** (*hidden layers*): están entre la entrada y la salida y no se conectan con el exterior de la red.[58] En el ejemplo, una capa de 15 neuronas;
  - **de salida** (*output layer*): envía la respuesta al exterior de la red. En el ejemplo, 10 neuronas, una por dígito.

Capa	Recibe de	Envía a	Ej. dígitos
Entrada	el exterior (el estímulo)	la primera capa oculta	784 (1 por píxel)
Oculta(s)	la capa anterior	la capa siguiente	15
Salida	la última capa oculta	el exterior (la respuesta)	10 (1 por dígito)

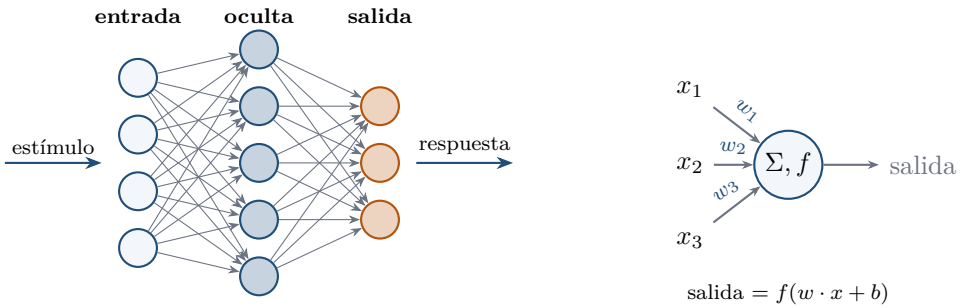


Figura 9: RNA de tres capas (izquierda) y detalle de una neurona artificial (derecha).

EXTRAS

- Ejemplo de la cátedra: un perceptrón con $w_1 = w_2 = -2$ y $b = 3$ implementa NAND:[59]

[55] Cátedra, Episodio VII, parte 3, «Redes Neuronales Artificiales».
[56] Cátedra, Episodio VII, parte 3, «Neuronas Artificiales».
[57] Cátedra, Episodio VII, parte 3, «Redes Neuronales Artificiales».
[58] Misma idea en AIMA, §18.7.1, p. 729.
[59] Cátedra, Episodio VII, parte 3, «Ejemplo».

x_1	x_2	$-2x_1 - 2x_2 + 3$	salida
0	0	3	1
0	1	1	1
1	0	1	1
1	1	−1	0

- Con conexión completa entre capas, la red de dígitos tiene $784 \times 15 + 15 \times 10 = 11.910$ arcos, y cada uno tiene un peso por ajustar.[60]
- Una función de activación no lineal (escalón o sigmoide) permite que la red represente funciones no lineales. Además, la sigmoide es **derivable**, con $f'(x) = f(x)\,(1 - f(x))$, y eso es lo que necesita el descenso de gradiente (4.b).[61]
- Las unidades ocultas no tienen un significado fijado de antemano: lo que representan es algo que se aprende, y a veces es difícil de interpretar.[62]
- Se inspiran en las neuronas del cerebro pero no las simulan: tienen muchas menos unidades que las $\approx 10^{11}$ neuronas humanas, y mucho más simples.[63]
- Antes de entrenar hay que elegir la forma de la red. La cátedra muestra varias, incluida la arquitectura *Transformer*.[64]

4.b

Consigna

¿En qué consiste el método de Backpropagation? ¿Cuál es su relación con el Descenso de Gradiente Estocástico?

- **El problema**: entrenar una RNA es un problema de **minimización**. Dado un conjunto de ejemplos, hay que encontrar los valores de los parámetros (w, b) que minimicen el error cuadrático de la salida.[65]
- **Descenso de gradiente estocástico (SGD)**: el método de **búsqueda** del mínimo, similar a *hill climbing*. Observa la pendiente de su vecindario local y se mueve en la dirección de pendiente más pronunciada.[66] Es *estocástico* porque considera un solo ejemplo de entrenamiento por vez y da un paso después de cada uno.[67]
- **Backpropagation**: el método para **calcular el gradiente** de la función de error.[68] Esquema:[69]
  1. se asignan valores iniciales al azar;
  2. se repite hasta un umbral definido:
     - se evalúa un ejemplo de entrenamiento **hacia adelante** (*forward*), de la entrada a la salida;
     - se calculan los errores **hacia atrás** (*backward*), de la salida a la entrada.
- **Por qué hacia atrás**: en la capa de salida el error se conoce (salida esperada menos obtenida), pero los datos no dicen qué valor deberían tener las neuronas ocultas. Por eso el error se

[60] Cátedra, Episodio VII, parte 3, «Redes Neuronales Artificiales».
[61] AIMA, §18.7.1, p. 729; PMG, §11.2, p. 410.
[62] PMG, §11.2, p. 414.
[63] PMG, §11.2, p. 408.
[64] Cátedra, Episodio VII, parte 3, «Antes del entrenamiento hay que elegir que RNA usar».
[65] Cátedra, Episodio VII, parte 3, «Neuronas Artificiales».
[66] Cátedra, Episodio VII, parte 3, «Neuronas Artificiales» y «Stochastic gradient descent vs. Hill climbing».
[67] AIMA, §18.6.1, p. 720.
[68] Cátedra, Episodio VII, parte 3, «Stochastic gradient descent vs. Hill climbing».
[69] Cátedra, Episodio VII, parte 3, «Backpropagation».

propaga desde la salida hacia las capas ocultas.[70]

- **Relación**: el SGD decide **cómo moverse** (un paso en contra del gradiente por cada ejemplo), y backpropagation calcula **el gradiente** que el SGD necesita en cada paso.

	Descenso de gradiente estocástico	**Backpropagation**
Qué es	método de búsqueda / optimización	método de cálculo del gradiente
Responde	¿hacia dónde y cuánto muevo los pesos?	¿cuánto influye cada peso en el error?
Cómo	un paso en la dirección de mayor descenso, por cada ejemplo	forward (salida) + backward (errores capa por capa)

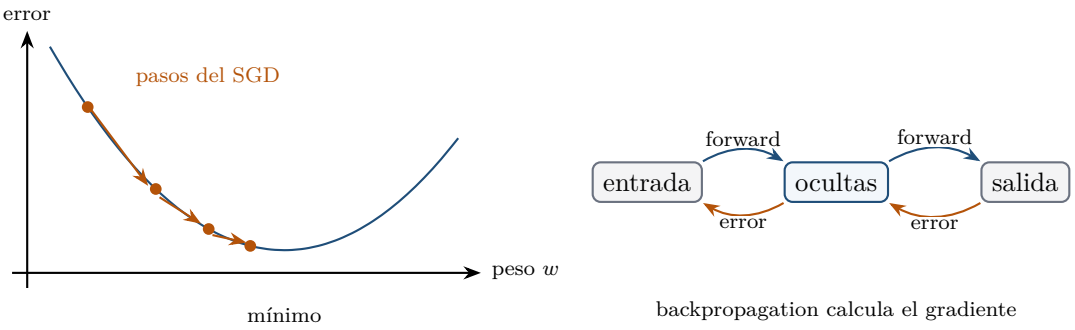


Figura 10: El SGD baja por la curva del error (izquierda) con el gradiente que calcula backpropagation (derecha).

EXTRAS

- **SGD vs. hill climbing**:[71]

	Hill climbing	**Descenso de gradiente**
Objetivo	maximiza	minimiza
Cómo elige	evalúa todos los vecinos y va al mejor	mira la pendiente local y sigue la más pronunciada
Problemas	discretos	funciones continuas con gradiente calculable
Eficiencia	menor	en general mucho mayor, pero sirve para menos problemas

- **Historia**: backpropagation se introdujo en la década de 1970, pero se valoró recién con el artículo de Rumelhart, Hinton y Williams (1986), que mostró que era mucho más rápido que los enfoques anteriores.[72]
- **Regla de actualización**: $w_{i,j} \leftarrow w_{i,j} + \alpha \cdot a_i \cdot \Delta_j$, donde Δ_j es el error "modificado" de la unidad j y α es la **tasa de aprendizaje** (*learning rate*). Cada peso cambia en proporción a cuánto reduce el error un pequeño cambio en él.[73]
- **Estocástico vs. por lotes** (*batch*): el descenso por lotes recorre todos los ejemplos antes de cada paso. El SGD suele ser más rápido, pero con α fijo puede oscilar alrededor del mínimo sin converger. Una tasa de aprendizaje decreciente, como en *simulated annealing*, garantiza la convergencia.[74]

[70] AIMA, §18.7.4, p. 733.
[71] Cátedra, Episodio VII, parte 3, «Stochastic gradient descent vs. Hill climbing».
[72] Cátedra, Episodio VII, parte 3, «Backpropagation».
[73] AIMA, §18.7.4, pp. 733–734; PMG, §11.2, p. 412.
[74] AIMA, §18.6.1, p. 720.

4.c

Consigna

Explique brevemente cómo se desarrolla el proceso de entrenamiento en una RNA. ¿Este proceso modifica las conexiones entre las neuronas? ¿Qué tipo de modificaciones hace?

Proceso. Pasos:[75]

1. **Antes de entrenar**: se elige la configuración de la red y la función matemática de cada nodo.
2. Se asignan **valores iniciales** (al azar) a los arcos.
3. Se usan los ejemplos etiquetados para ir **adecuando los pesos** w de cada arco: por cada ejemplo, forward, cálculo del error y backpropagation con SGD (Ejercicio 4.b).
4. Se repite hasta que la red se comporte con un **error aceptable** (p. ej. 97 % de aciertos).

Que la red esté **entrenada** significa que se encontraron valores adecuados para todos los pesos w_i de los arcos y los umbrales de cada nodo, de modo que la red haga "buenas" predicciones.[76]

¿Modifica las conexiones? No modifica qué conexiones hay, sino cuánto vale cada una:[77]

- **No cambia la topología**: qué neurona se conecta con cuál se fija antes de entrenar, al elegir la configuración de la red.
- **Sí cambia el valor de cada conexión**: ajusta numéricamente los pesos w de los arcos (y los umbrales/sesgos b de los nodos).
- El peso determina la **intensidad** y el **signo** de la conexión, así que el entrenamiento cambia cuánto y en qué sentido influye cada neurona sobre la siguiente.[78]

Se fija antes de entrenar	Cambia durante el entrenamiento
cantidad de capas y de neuronas	valor de cada peso w (intensidad y signo)
qué neuronas se conectan (arcos)	umbral / sesgo b de cada neurona
función de cada nodo (perceptrón, sigmoide)	

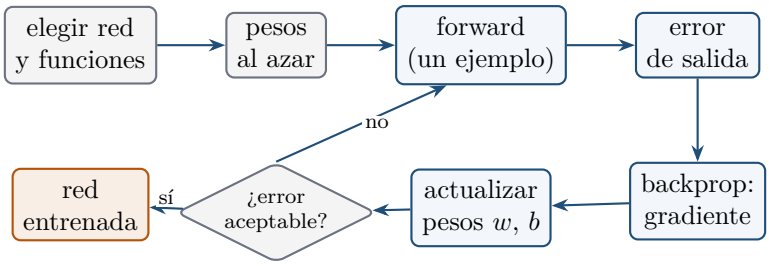


Figura 11: Ciclo de entrenamiento de una RNA: la topología no cambia, solo los pesos.

EXTRAS

- Una vez conformes con el entrenamiento, para usar la red solo hace falta la red con los pesos de los arcos.[79]

[75] Cátedra, Episodio VII, parte 3, «Redes Neuronales Artificiales» (¿Cómo se entrena una RNA?) y «Backpropagation».
[76] Cátedra, Episodio VII, parte 3, «Redes Neuronales Artificiales».
[77] Cátedra, Episodio VII, parte 3, «Redes Neuronales Artificiales».
[78] AIMA, §18.7.1, p. 728.
[79] Cátedra, Episodio VII, parte 3, «Redes Neuronales Artificiales».

- Con varias salidas, el gradiente de la pérdida es la suma de los gradientes de cada salida. Un problema de m salidas se descompone en m problemas, sumando sus contribuciones al actualizar los pesos.[80]
- PMG estima las derivadas simulando la red con cada parámetro levemente cambiado. Para la sigmoide aplicada a una combinación lineal, la derivada se calcula analíticamente, y así el algoritmo corre más rápido.[81]

4.d

Consigna

Describa una Red Neuronal Feed-Forward (hacia adelante) en cuanto a su topología y flujo de señales.

La cátedra la usa (la red de dígitos y el paso *feedforward* de backpropagation) pero no la define.[82] Según AIMA:[83]

- **Topología**:
  - las conexiones van en **una sola dirección**: es un **grafo dirigido acíclico**, sin ciclos;
  - en general se organiza en **capas**, y cada unidad recibe entradas solo de la capa inmediatamente anterior.
- **Flujo de señales**:
  - la señal entra por la capa de entrada, atraviesa las ocultas y sale por la de salida: cada nodo recibe de los nodos anteriores (*upstream*) y entrega a los siguientes (*downstream*);
  - como no hay ciclos, la salida es **función solo de la entrada actual**: la red no tiene estado interno (memoria) aparte de los pesos.

Aspecto	Red feed-forward
Grafo	dirigido acíclico (sin ciclos)
Organización	en capas; cada unidad recibe solo de la capa anterior
Flujo	entrada → ocultas → salida, siempre hacia adelante
Salida	depende solo de la entrada actual (sin memoria más que los pesos)

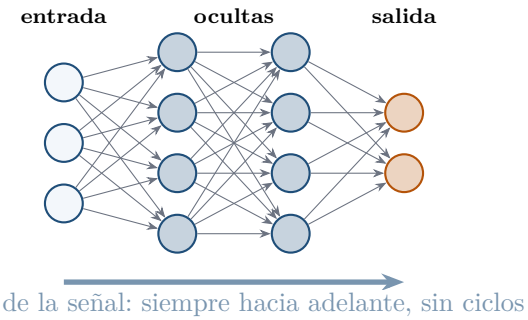


Figura 12: Red feed-forward con dos capas ocultas.

EXTRAS

[80] AIMA, §18.7.4, p. 733.
[81] PMG, §11.2, p. 412.
[82] Cátedra, Episodio VII, parte 3, «Backpropagation».
[83] AIMA, §18.7.1, p. 729.

- **Redes recurrentes**, la alternativa: realimentan sus salidas hacia sus propias entradas. Forman un sistema dinámico que puede estabilizarse, oscilar o tener comportamiento caótico. La respuesta depende del estado anterior, así que tienen memoria de corto plazo: se parecen más al cerebro, pero son más difíciles de entender.[84]



- **Una capa vs. multicapa**: si todas las entradas se conectan directo a las salidas, es una red de una capa (red de perceptrones); si tiene una o más capas ocultas, es multicapa.[85]
- PMG compara las redes feed-forward con programas lógicos de encadenamiento hacia adelante, sin ciclos, con parámetros por aprender.[86]

---

[84] AIMA, §18.7.1, p. 729.
[85] AIMA, §18.7.1, p. 729.
[86] PMG, §11.2, p. 409.